

Sistema IoT Industrial Escalable y de Bajo Costo para la Monitorización de Tiempos de Actividad en Maquinaria Industrial

Arquitectura de Servidor, Motor de Actividades Automatizado e Interfaces Duales de Gestión

Hernán F. Kisiel¹, Germán A. Xander², Luis A. Urbani³, Alejandro G. Maxit⁴

Universidad Nacional de Misiones, Facultad de Ingeniería
Oberá, Misiones, Argentina

E-mails: hernankisiel@gmail.com, german.xander@fio.unam.edu.ar,
ctt.urbani@gmail.com, alejandro.maxit@fio.unam.edu.ar

31 de julio de 2026

Resumen

En la era de la Industria 4.0, la digitalización de plantas de producción con maquinaria convencional representa un desafío crítico para las pequeñas y medianas empresas (PyMEs) debido a los elevados costos de inversión en sistemas SCADA o plataformas MES propietarias. Este artículo presenta la investigación, diseño e implementación end-to-end de un sistema de Internet de las Cosas (IoT) industrial de bajo costo, fácil instalación, alta usabilidad y arquitectura fuertemente escalable. La captura de eventos operativos se realiza en el borde mediante un módulo sensor no invasivo con aislamiento galvánico optoelectrónico conectado a los contactores de 24V de maquinaria electromecánica (torno y fresadora), procesado por microcontroladores ESP32 y transmitido vía MQTT. Toda la infraestructura de servidor se despliega mediante contenedores Docker orquestados con Docker Compose, integrando un proxy inverso SWAG (Nginx HTTPS), un broker MQTT Mosquitto, un servicio de ingesta en Python, MariaDB, una API REST en FastAPI, un Bot conversacional en Telegram 2.0 y una aplicación web SPA en HTML5/Bootstrap 5. Se destaca el desarrollo de un motor de actividades automatizado capaz de consolidar la telemetría en sesiones continuas de trabajo, gestionar temporizadores de inactividad, calcular el desgaste acumulado de herramientas y proveer herramientas de ajuste retroactivo (Merge y Split). El sistema fue validado en planta acumulando 30 días de funcionamiento continuo con 134 actividades generadas, demostrando ser una solución robusta y accesible para la supervisión industrial.

Palabras Clave: IoT Industrial, ESP32, Arquitectura de Servidor, FastAPI, Docker, MQTT,

Telegram Bot, Web Admin SPA, Industria 4.0, Gestión de Maquinaria.

1. Introducción y Planteamiento del Problema

En las pequeñas y medianas empresas (PyMEs) del sector metalúrgico, la eficiencia global de los equipos (OEE) y la optimización de los costos operativos dependen de manera directa de la maximización del tiempo de uso efectivo de maquinaria crítica, como tornos, fresadoras y rectificadoras. Sin embargo, la gestión de la producción en estas plantas suele caracterizarse por una marcada falta de visibilidad en tiempo real. Tradicionalmente, la supervisión de los tiempos de parada y de trabajo se ha basado en la presencia física continua del supervisor o en planillas manuales completadas por los propios operarios al finalizar la jornada. Estos métodos convencionales presentan deficiencias estructurales: son propensos a sesgos humanos, inconsistencias temporales, pérdidas de datos y retrasos significativos en la detección de ineficiencias o averías.

Por otro lado, la inmensa mayoría de las máquinas instaladas en las PyMEs metalúrgicas corresponden a equipos convencionales electromecánicos con décadas de servicio útil acumulado. Estas máquinas operan mediante tableros eléctricos basados en contactores, relés y pulsadores alimentados a 24V de corriente continua o alterna, careciendo por completo de sensores digitales integrados, controladores lógicos programables (PLC) o puertos de comunicación de datos. Si bien existen soluciones comerciales de digitalización industrial (como sistemas SCADA dedicados o plataformas MES empresariales), su implementación resulta inviable para este segmento productivo. Los costos de licenciamiento de software, la necesidad de reemplazar componentes industriales propietarios y los honorarios de integración superan con frecuencia los varios miles de dólares por máquina, requiriendo además largos períodos de parada de planta para su instalación.

Para superar estas limitaciones, el presente trabajo tiene como objetivo el desarrollo y validación de una solución tecnológica IoT integral basada en cuatro pilares de diseño:

1. **Bajo Costo de Adquisición y Mantenimiento:** Utilización exclusiva de hardware de desarrollo abierto de bajo costo (microcontroladores ESP32 y componentes discretos) combinado con una pila de software basada 100 % en plataformas Open Source sin costo de licenciamiento.
2. **Fácil Instalación y No Invasividad:** Captación de señales mediante circuitos de acondicionamiento optoelectrónicos que toman lectura directamente de las bobinas de los contactores de 24V sin cortar cables, modificar la lógica de seguridad original ni requerir la intervención de los circuitos de potencia de la máquina.
3. **Fácil Uso e Interfaces Duales:** Provisión de herramientas digitales adaptadas al perfil del usuario final. Para el operario de planta, se implementa un Bot conversacional interactivo en Telegram 2.0 que no requiere instalar aplicaciones adicionales; para la gerencia y supervisión, se despliega un panel Web Admin SPA (Single Page Application) accesible desde cualquier navegador web.
4. **Alta Escalabilidad Horizontal:** Arquitectura orientada a microservicios independientes contenedorizados mediante Docker, lo que permite escalar el sistema para incorporar decenas de máquinas y sensores adicionales simplemente duplicando los nodos de borde sin reestructurar el servidor central.

2. Descripción del Sensor y Acondicionamiento Hardware

El segmento sensor (Edge Hardware) cumple la función de detectar y transmitir en tiempo real las variaciones en el estado operativo de los motores principales de la maquinaria.

En el caso del torno industrial analizado, el circuito de control de planta cuenta con dos pulsadores momentáneos (uno de arranque y uno de parada) que energizan la bobina del contactor principal de 24V. En la fresadora universal, el encendido y selección de sentido de giro del husillo se realiza mediante un selector rotativo de levas de tres posiciones (Izquierda–Apagado–Derecha).

2.1. Acondicionamiento de Señal y Aislamiento Galvánico

Para interconectar las señales eléctricas del tablero industrial (24V) con las entradas digitales de bajo voltaje del microcontrolador ESP32 (3.3V), se diseñó un módulo de acondicionamiento de señal con aislamiento galvánico basado en optoacopladores de alta velocidad (PC817).

Cuando el operador acciona el arranque y la bobina del contactor recibe 24V, se activa el diodo emisor del optoacoplador, saturando el fototransistor de salida. Esto conmuta a masa una línea de entrada digital del ESP32 configurada con resistencia de *pull-up* interna, generando un nivel lógico bajo (0V) perfectamente aislado de cualquier pico de tensión, ruido inductivo o transitorio proveniente de los motores eléctricos de alta potencia.

2.2. Lógica de Borde y Firmware ESP32

Cada máquina dispone de un nodo ESP32 independiente alojado en una caja estanca industrial IP65. El firmware del dispositivo está desarrollado sobre el framework Arduino/ESP-IDF e incluye las siguientes funciones clave:

- **Filtro Anti-rebote (Debounce):** Algoritmo de filtrado por software de 50 ms para eliminar conmutaciones falsas producidas por la vibración mecánica de los contactos eléctricos.
- **Detección de Eventos Lógicos:** Discriminación entre estados de operación normal (ACTIVA, PARADA) y estados de falla o interrupción intencional de seguridad (PARADA_EMERGENCIA).
- **Conectividad Wi-Fi y Protocolo MQTT:** Publicación inmediata de mensajes JSON livianos ante cada cambio de estado hacia el broker MQTT central utilizando la biblioteca PubSubClient.

3. Arquitectura Completa de Servidor e Infraestructura

La capa de servidor representa el componente central del proyecto, responsable de la ingesta masiva de datos, el almacenamiento relacional, la ejecución de la lógica de negocio automatizada y el suministro de las interfaces visuales para la toma de decisiones. Toda la pila de software se orquesta de manera centralizada mediante Docker Compose sobre un sistema operativo Linux.

3.1. Microservicios Orquestados en Docker

La infraestructura se compone de ocho microservicios desacoplados que ejecutan dentro de una red virtual interna aislada:

- **SWAG (Secure Web Application Gateway):** Proxy inverso basado en Nginx que actúa como única puerta de entrada al servidor desde el exterior, gestionando certificados de cifrado SSL/TLS (HTTPS) en los puertos 80 y 443.
- **Mosquitto (MQTT Broker):** Broker de mensajería ligero de alto rendimiento que recibe las publicaciones de los dispositivos ESP32 en el puerto 1883 (interno) y 8883 (seguro).
- **Cliente MQTT Ingesta (Python Service):** Servicio en segundo plano que mantiene una suscripción continua al tópico `industrial/metalurgica/+/estado`. Al recibir un mensaje, interpreta el payload, valida el timestamp y registra el evento en la base de datos.
- **MariaDB:** Motor de base de datos relacional altamente eficiente optimizado para la persistencia de transacciones, telemetría y consolidación de KPIs.
- **FastAPI REST Backend (API IoT):** Servidor de aplicaciones web asíncrono (ASGI) desarrollado en Python 3.11 que expone la API RESTful del sistema, sirve los archivos estáticos de la interfaz web y ejecuta los algoritmos de ajuste retroactivo de actividades.
- **Bot de Telegram 2.0:** Servicio conversacional escrito en Python mediante la librería `python-telegram-bot` que permite la interacción operativa con el personal de planta.
- **Grafana Analytics:** Plataforma de visualización de métricas en tiempo real conectada directamente a MariaDB para la generación de tableros ejecutivos.
- **phpMyAdmin:** Interfaz web de administración de base de datos para tareas de mantenimiento técnico.

4. Motor de Actividades Automatizado y Lógica de Telemetría

Uno de los principales aportes del sistema radica en el desarrollo del **Motor de Actividades Automatizado**. Los sistemas de monitoreo simples se limitan a guardar registros aislados de timestamps cada vez que una máquina se enciende o apaga. Esto genera miles de filas inconexas que dificultan conocer el tiempo real de trabajo efectivo por pieza o por operario.

El motor de actividades soluciona esto agrupando inteligente y automáticamente los eventos de telemetría en **sesiones continuas de trabajo** (denominadas actividades), asociándolas al operario que ha fichado en la máquina y a la herramienta de corte que se encuentra montada.

4.1. Lógica de Consolidación y Temporizador $T_{\text{inactividad}}$

El algoritmo de consolidación opera bajo las siguientes reglas de transición:

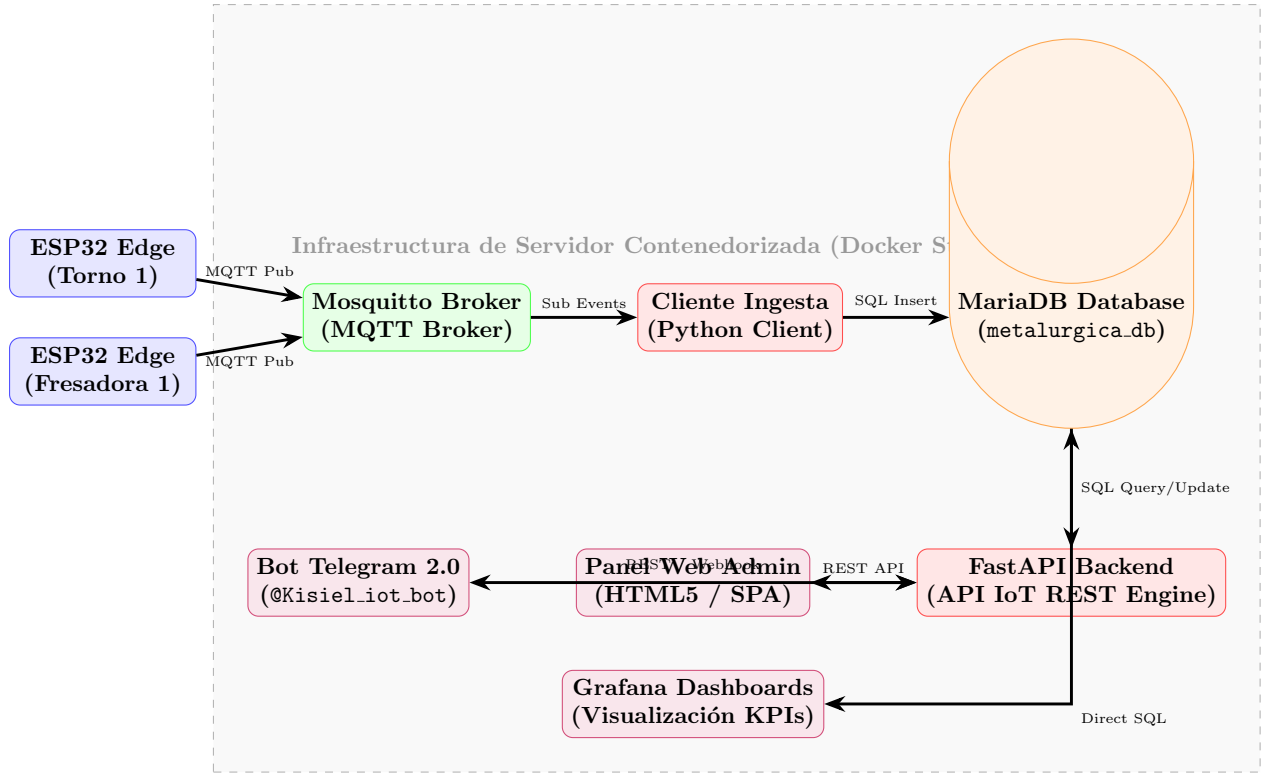


Figura 1: Diagrama de bloques general de la arquitectura de servidor y flujo de datos end-to-end.

1. **Inicio de Actividad (ACTIVA):** Cuando el cliente de ingesta recibe una señal de arranque, verifica si la máquina posee una actividad registrada en estado `EN_CURSO`. Si ya existe, se mantiene abierta acumulando tiempo. Si no existe, consulta la tabla de asignaciones actuales (`asignaciones_actuales`) para obtener el ID del operario y de la herramienta, e inserta una nueva fila en la tabla `actividades` marcando la hora exacta de inicio (`fecha_inicio`).
2. **Manejo de Micro-paradas e Inactividad ($T_{\text{inactividad}}$):** Cuando la máquina se detiene (`PARADA`), el motor no cierra la actividad de inmediato. En los procesos metalúrgicos es habitual que un operario detenga el torno durante unos minutos para medir una pieza con el calibre, cambiar la velocidad de la caja de engranajes o colocar lubricante. El sistema inicia un temporizador de inactividad configurable ($T_{\text{inactividad}}$, por defecto 10 minutos).
 - Si la máquina vuelve a encenderse dentro del margen $T_{\text{inactividad}}$, el temporizador se cancela y la actividad continúa abierta sin interrupción.
 - Si la máquina permanece apagada superando el límite $T_{\text{inactividad}}$, el motor da la sesión por concluida automáticamente, actualizando su estado a `FINALIZADA`, fijando la `fecha_fin` en la hora que comenzó la parada y calculando el tiempo activo neto acumulado (`tiempo_activo_segundos`).
3. **Interrupción por Parada de Emergencia:** Si el microcontrolador emite el código de `PARADA_EMERGENCIA`, la actividad en curso se finaliza de forma instantánea bajo dicho estado y se genera una notificación de alerta en tiempo real.

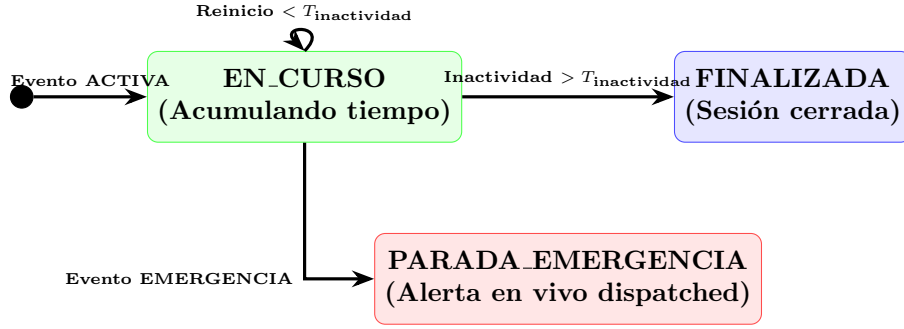


Figura 2: Diagrama de estados de la lógica de consolidación de actividades.

4.2. Módulo de Ajustes Retroactivos (Fit de Actividades)

En la práctica industrial real, ocurren situaciones atípicas: un operario puede olvidar fichar al iniciar su turno, o un trabajo continuo de varias horas puede ser interrumpido por el cambio de turno. Para mantener la precisión estadística, la API en FastAPI incluye tres algoritmos de ajuste retroactivo:

- **Fusión (Merge):** Permite al administrador seleccionar dos o más actividades consecutivas erróneamente fragmentadas y combinarlas en un único registro, unificando los timestamps de inicio y fin y sumando los tiempos activos netos.
- **División (Split):** Permite dividir una actividad en dos fragmentos especificando una fecha y hora de corte exacta. El primer fragmento conserva el inicio original y se cierra en la hora de corte; el segundo fragmento se inicia en la hora de corte y hereda la hora de fin original.
- **Asignación Retroactiva de Operario:** Permite vincular un operario a sesiones de trabajo que fueron registradas de forma anónima.

5. Esquema Relacional de Base de Datos

La persistencia de datos del sistema está encomendada a MariaDB mediante la base de datos `metalurgica_db`. El modelo entidad-relación ha sido diseñado bajo principios de normalización para asegurar la coherencia de los datos y optimizar la velocidad de respuesta en consultas de tableros de control.

El esquema relacional consta de ocho tablas principales interconectadas mediante claves foráneas y restricciones de integridad referencial, detalladas en la Tabla 1.

6. Interfaces de Gestión y Control

Con el fin de garantizar una adopción exitosa sin barreras de aprendizaje, el sistema provee dos interfaces de usuario independientes construidas sobre el backend de FastAPI.

Tabla 1: Especificación Detallada del Esquema Relacional de Base de Datos (*metalurgica_db*)

Tabla	Clave Primaria	Claves Foráneas	Propósito Operativo y Campos
<code>maquinas</code>	<code>id</code> (INT)	N/A	Registro inventariado de activos. Campos: <code>nombre</code> , <code>tipo</code> , <code>ubicacion</code> .
<code>usuarios</code>	<code>id</code> (INT)	N/A	Credenciales y vinc. Telegram. Campos: <code>nombre</code> , <code>username</code> , <code>rol</code> , <code>activo</code> .
<code>herramientas</code>	<code>id</code> (INT)	<code>maquina_id</code>	Desgaste acumulado de herramientas. Campos: <code>nombre</code> , <code>horas_expectativa</code> .
<code>asignaciones_actuales</code>	<code>maquina_id</code>	<code>operario_id</code> , <code>herramienta_id</code>	Estado instantáneo del taller. Indexa operario y herramienta por máquina.
<code>registro_actividad</code>	<code>id</code> (BIGINT)	<code>maquina_id</code>	Historial inmutable de telemetría. Campos: <code>estado</code> , <code>codigo_estado</code> , <code>timestamp</code> .
<code>actividades</code>	<code>id</code> (INT)	<code>maquina</code> , <code>operario</code> , <code>herramienta</code>	Sesiones consolidadas. Campos: <code>fecha_inicio</code> , <code>fecha_fin</code> , <code>tiempo_activo_seg</code> , <code>estado</code> .
<code>kpi_turnos</code>	<code>id</code> (INT)	<code>maquina_id</code>	Métricas por turno. Campos: <code>fecha</code> , <code>turno</code> , <code>tiempo_activo_seg</code> , <code>disponibilidad_pct</code> .
<code>configuracion_sistema</code>	<code>clave</code>	N/A	Parámetros globales. Campos: <code>valor</code> , <code>descripcion</code> .

6.1. Panel Web Admin SPA (FastAPI + Bootstrap 5)

El Panel de Administración Web es una aplicación de página única (SPA) desarrollada con tecnología estándar web (HTML5, Vanilla JavaScript asíncrono y Bootstrap 5), servida directamente por FastAPI en la URL `http://servidor/admin`. Presenta un diseño estético industrial en modo oscuro.

La aplicación organiza sus funciones en seis secciones principales:

- Dashboard de Monitoreo en Tiempo Real:** Muestra tarjetas dinámicas por cada máquina con refresco automático cada 5 segundos. Permite visualizar instantáneamente si la máquina está **ACTIVA**, **PARADA** o en **PARADA EMERGENCIA**, exhibiendo el nombre del operario en turno y la herramienta montada.
- Gestión de Operarios:** Permite registrar nuevos operarios, realizar bajas lógicas y consultar el resumen individual de desempeño mediante un modal emergente que genera gráficos estadísticos dinámicos de barras por turno (Matplotlib).
- Gestión de Herramientas y Vida Útil:** Presenta una tabla interactiva con barras de progreso de desgaste dinámicas codificadas por colores: **Verde** ($< 70\%$), **Amarillo** ($70 - 90\%$) y **Rojo** ($> 90\%$), calculadas automáticamente según el tiempo de trabajo acumulado sobre la expectativa de vida útil.
- Control de Actividades (Fit Retroactivo):** Permite seleccionar casillas múltiples para ejecutar fusiones (*Merge*), abrir el modal de división (*Split*) o corregir la asignación de operarios.

5. **Informes y KPIs Consolidados:** Muestra reportes consolidados por turno y gráficos visuales (barras de uso diario y gráfico de rosquilla de utilización semanal) junto con un botón para la descarga directa del historial completo en archivo CSV.
6. **Configuración del Sistema:** Formulario para modificar los límites de inactividad ($T_{\text{inactividad}}$) y las horas de inicio/fin de turnos de trabajo.

6.2. Bot Conversacional de Telegram 2.0 (@Kisiel_iot_bot)

Para el trabajo diario en la nave de producción, la API de Telegram provee una interfaz ligera y accesible desde cualquier teléfono inteligente o tablet sin necesidad de instalar software propietario.

Las características principales del Bot 2.0 son:

- **Autenticación Basada en Roles (RBAC):** El usuario se autentica mediante el comando `/login` ingresando su usuario y clave cifrada. El sistema asigna los permisos correspondientes según el rol del usuario (ADMIN u OPERARIO).
- **Fichaje mediante Botoneras Lógicas (Inline Keyboards):** El operario puede indicar qué máquina va a utilizar y qué herramienta ha montado presionando botones interactivos en pantalla, sin necesidad de escribir texto.
- **Diseño Limpio Sin Emojis:** La interfaz utiliza un formato de texto industrial sumamente claro basado en corchetes e identificadores (`[Gestión Operarios]`, `[Estado Máquinas]`, `[Cerrar Sesión]`), optimizado para una lectura veloz en entornos de baja iluminación o pantallas pequeñas.
- **Alerta Inmediata de Parada de Emergencia:** Ante una parada por botón de emergencia en cualquier máquina, el backend utiliza la API de Telegram para enviar una alerta prioritaria instantánea a todos los administradores registrados.

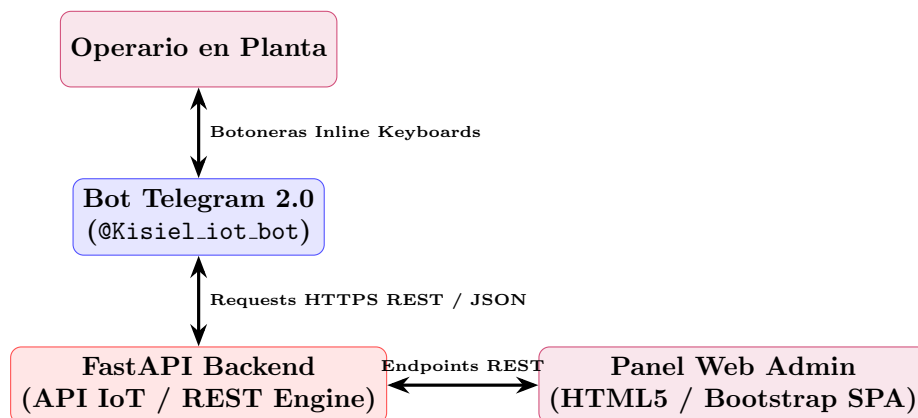


Figura 3: Diagrama de secuencia e interacción entre los usuarios y el servidor central.

7. Evaluación de Resultados, Escalabilidad y Análisis de Costos

El sistema desarrollado fue instalado y probado en una planta metalúrgica real, acumulando un período de evaluación continua de 30 días con una carga de 134 sesiones de actividad generadas entre un torno industrial y una fresadora universal.

7.1. Análisis Económico Comparativo

Un aspecto diferencial del proyecto es su extrema viabilidad económica. Las soluciones comerciales de telemetría o adquisición de datos industriales suelen requerir módulos de entrada/salida Ethernet propietarios, licencias anuales de software SCADA por tag y costos de implementación que superan fácilmente los 1.500 USD por máquina.

En contraste, la solución propuesta presenta los siguientes costos de hardware por nodo de máquina:

- Microcontrolador ESP32 DevKit v1: \$6.00 USD.
- Módulo de optoacopladores de 4 canales PC817: \$3.50 USD.
- Fuente de alimentación conmutada 220V/5V 2A: \$5.00 USD.
- Gabinete estanco IP65, conectores e instalación: \$12.00 USD.
- **Costo Total de Hardware por Máquina: \$27.50 USD.**

Dado que el servidor opera íntegramente sobre software libre en contenedores Docker, los costos de software y licencias continuas son nulos.

7.2. Despliegue Automatizado y Escalabilidad

El despliegue de la plataforma en un nuevo servidor de planta se encuentra 100 % automatizado mediante Docker Compose. Ejecutando el comando `docker compose up -d`, el sistema descarga, construye y pone en marcha los ocho microservicios en menos de tres minutos.

En términos de escalabilidad horizontal, la arquitectura basada en el patrón Publicador–Suscriptor (Pub-Sub) sobre MQTT permite al broker Mosquitto procesar miles de mensajes por segundo con una utilización de memoria RAM inferior a 15 MB. Para incorporar una nueva máquina al sistema, únicamente se requiere instalar su nodo ESP32 correspondiente y dar de alta el nombre de la máquina desde la interfaz Web Admin.

8. Conclusión

El presente trabajo ha demostrado la factibilidad técnica, operativa y económica de digitalizar maquinaria convencional de la industria metalúrgica mediante un enfoque IoT moderno, de bajo costo y altamente escalable.

La combinación de captadores optoelectrónicos no invasivos en el borde con una infraestructura de servidor microserviciada orquestada en Docker permite transformar datos lógicos brutos de contactores de 24V en información estratégica de alto valor: sesiones continuas de trabajo,

indicadores clave de rendimiento (KPIs) por turno, seguimiento de la vida útil de herramientas de corte y alertas automáticas de seguridad.

Asimismo, la provisión de interfaces de usuario adaptadas (un Bot conversacional en Telegram 2.0 para operarios y un Panel Web Admin SPA para la gerencia) resuelve de manera definitiva la falta de visibilidad operativa en la planta sin imponer barreras de aprendizaje ni costos recurrentes de licenciamiento.

Referencias

- [1] Espressif Systems, “ESP32 Technical Reference Manual,” version 4.8, 2023. [En línea]. Disponible en: <https://www.espressif.com>
- [2] OASIS Standard, “MQTT Version 5.0 Standard Specification,” 2019. [En línea]. Disponible en: <https://mqtt.org>
- [3] Docker Inc., “Enterprise Application Container Platform and Docker Compose Documentation,” 2024. [En línea]. Disponible en: <https://www.docker.com>
- [4] S. Ramírez, “FastAPI Framework Architecture and Performance Guide,” 2024. [En línea]. Disponible en: <https://fastapi.tiangolo.com>
- [5] MariaDB Foundation, “MariaDB Enterprise Database Documentation,” 2024. [En línea]. Disponible en: <https://mariadb.org>
- [6] Telegram Messenger Inc., “Telegram Bot API and Inline Keyboards Documentation,” 2024. [En línea]. Disponible en: <https://core.telegram.org/bots/api>
- [7] Bootstrap Core Team, “Bootstrap 5 Frontend Framework Guide,” 2024. [En línea]. Disponible en: <https://getbootstrap.com>